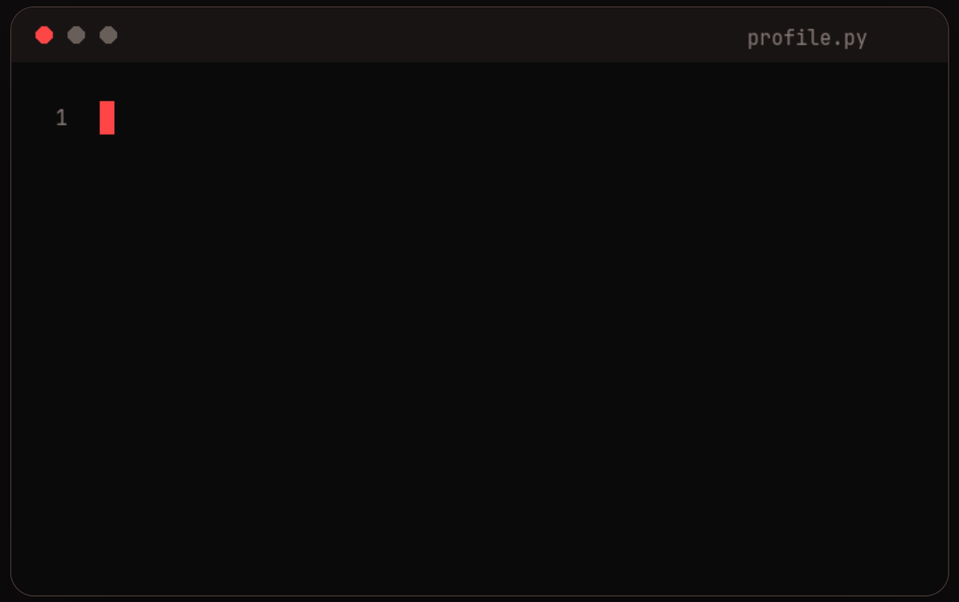
turknet

2026-27 GÜZ DÖNEMİ · HAFTA 2

Injection ve path traversal

SQL injection ve command injection
Path traversal



Can Öztaş

github.com/canoztas/secure-coding

BUGÜN

Ana konular

01	**SQL injection**	id parametresinin SQL sorgusunun parçası olması; parameterized query.
02	**Command injection**	Shell'e giden komut metni; argüman listesiyle çalıştırma.
03	**Path traversal**	../ ile dizin dışına çıkmak; dosyayı açmadan önce yol kontrolü.

Önce kısa bir threat modeling hatırlatması. Ekstra konular (süre kalırsa): argument injection, SSTI, NoSQL injection, XXE.

THREAT MODELING · INPUT

Input nereden gelir?

URL

Query string

/profile?id=1

Kullanıcı id değerini değiştirebilir.

FORM

Request body

name=alice

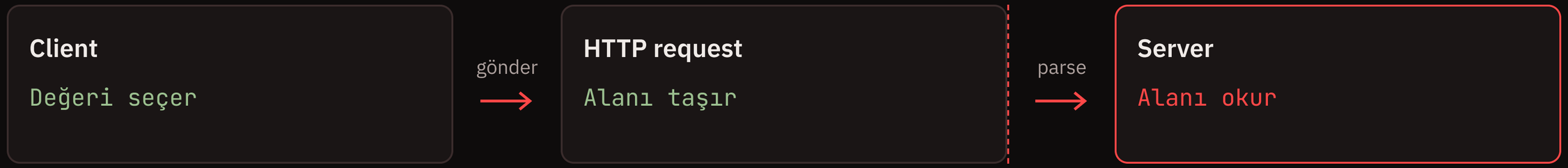
Form alanları body'de gönderilir.

JSON

API isteği

{"name": "alice"}

Tarayıcı dışındaki bir client da gönderebilir.



Client-side kontroller server-side validation'ın yerini tutmaz.

LAB · NORMAL REQUEST

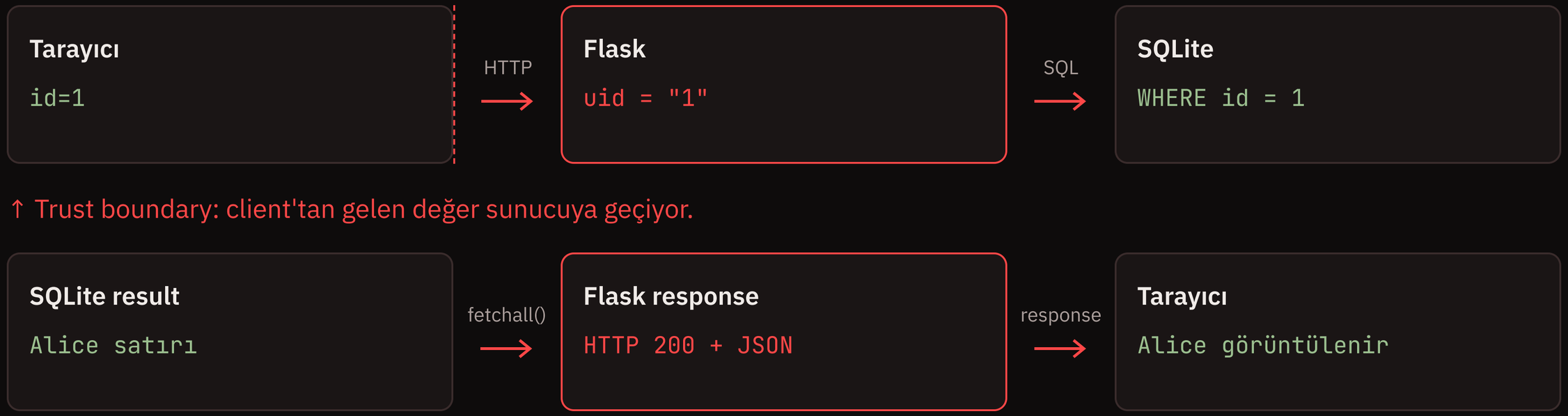
Uygulamanın normal davranışı



id=1: sunucu Alice kaydını döndürüyor.

THREAT MODELING · DATA FLOW

1 değeri hangi yolu izliyor?



Request'i almak, sorguyu çalıştırmak ve response üretmek ayrı adımlardır.

THREAT MODELING · DÖRT SORU

Shostack’ın dört sorusu

1

Ne yapıyoruz?

id → profil

2

Ne ters gidebilir?

id → sorgu değişir

3

Ne yapacağız?

Sabit SQL + parametre

4

İşe yaradı mı?

İki girdiyi yeniden dene

Adam Shostack · Threat Modeling: Designing for Security

THREAT MODELING · STRIDE

Tehditleri sınıflandırmak

S · SPOOFING

Kimlik taklidi

Başka kullanıcı gibi davranma

T · TAMPERING

Değiştirme

Sorgu koşulunu değiştirme

R · REPUDIATION

İnkâr

İşlem kaydının bulunmaması

I · INFORMATION DISCLOSURE

Veri açığa çıkması

Tek profil yerine tüm kayıtlar

D · DENIAL OF SERVICE

Hizmet engelleme

Pahalı sorguyla sistemi yorma

E · ELEVATION OF PRIVILEGE

Yetki yükseltme

Normal hesapla admin işlemi

BÖLÜM 1

SQL injection

Aynı /profile endpoint'i, aynı iki kayıt. Her adımda tek bir şey değişiyor.



SQL INJECTION · ÖRNEK VERİ

users tablosu

01 / 09 Uygulamanın verisi

id	name	email
1	alice	alice@example.test
2	bob	bob@example.test

query.sql

```
1  SELECT name, email
2  FROM users
3  WHERE id = 1
```

BEKLENEN SONUÇ

Yalnız Alice

SELECT: hangi kolonlar?
FROM: hangi tablo?
WHERE: hangi satırlar?

Tabloda iki kayıt var; parola ve session bu örnekte yok.

SQL INJECTION · REQUEST → DEĞİŞKEN

URL’deki id’yi hangi satır okuyor?

02 / 09 HTTP parametresi → Python değişkeni

request.http

```
GET /profile?id=1 HTTP/1.1
```

app.py

```python
@app.get("/profile")
def profile():
    uid = request.args["id"]
```

DEĞİŞKEN

uid = "1"

type: str

Flask /profile isteğini bu handler'a yönlendirir. id değeri HTTP'den her zaman string olarak gelir.

SQL INJECTION · DEĞİŞKEN → SORGU

Sorgu string birleştirmeyle kuruluyor

03 / 09 Sorgu metni çalıştırılmadan önce

app.py

```
1  query = "SELECT name, email FROM users "
2  query += "WHERE id = " + uid
```

Sabit SQL

`WHERE id =`

\+ →

Input

`uid = "1"`

concat →

Sonuç string

`WHERE id = 1`

SQLite'a gönderilecek sorgu

```
SELECT name, email FROM users WHERE id = 1
```

WHERE koşulu her satır için değerlendirilir

04 / 09 Veritabanının seçtiği satırlar

query.sql

```
1 SELECT name, email FROM users WHERE id = 1
```

Kayıt	Koşul	Seçilir mi?
Alice · id=1	1 = 1 → true	Evet
Bob · id=2	2 = 1 → false	Hayır

Result set · 1 kayıt

```
alice | alice@example.test
```

SQL INJECTION · RESULT SET → RESPONSE

Satırlar JSON response'a dönüşüyor

05 / 09 Sonucu okuyan ve döndüren satırlar

app.py

```python
uid = request.args["id"]
query = ("SELECT name, email FROM users "
         "WHERE id = " + uid)
rows = db.execute(query).fetchall()
return jsonify([dict(row) for row in rows])
```

response.http

```http
HTTP/1.1 200 OK
Content-Type: application/json

[
  {"name": "alice",
   "email": "alice@example.test"}
]
```

execute
Sorguyu çalıştırır

cursor →

fetchall
Satırları alır

list →

jsonify
JSON response üretir

db önceden açılmış bir sqlite3 bağlantısı; row_factory = sqlite3.Row.

SQL INJECTION · INPUT'U DEĞİŞTİR

1 yerine: 1 OR 1=1

06 / 09 Kod ve veri aynı, yalnız input farklı

request.http

```
1 GET /profile?id=1%20OR%201%3D1 HTTP/1.1
```

DEĞİŞKEN

uid = "1 OR 1=1"

type: str

URL ENCODING

Flask decode eder

%20 → boşluk

%3D → =

request.args decode edilmiş değeri verir.

Aynı birleştirmenin ürettiği sorgu

```
SELECT name, email FROM users WHERE id = 1 OR 1=1
```

SQL INJECTION · SORGUNUN ANLAMI DEĞİŞTİ

OR artık bir SQL operatörü

07 / 09 Yeni koşulu iki kayıt için değerlendir

query.sql

```
1  SELECT name, email FROM users
2  WHERE id = 1 OR 1=1
```

Kayıt	id = 1	1 = 1	OR sonucu
Alice	true	true	true → seçilir
Bob	false	true	true → seçilir

Input sorgu metnine eklendiği için SQL parser OR 1=1'i WHERE koşulunun parçası olarak okur.

LAB · SQL INJECTION

Aynı endpoint iki kayıt döndürüyor



Aynı veritabanı ve handler; yalnız id değeri değişti.

SQL INJECTION · FIX

Parameterized query

08 / 09 Sabit SQL + ayrı parametre listesi

app.py · /safe/profile

```
1  query = "SELECT name, email FROM users WHERE id = ?"
2  rows = db.execute(query, (uid,)).fetchall()
3  return jsonify([dict(row) for row in rows])
```

QUERY

Sabit kalır

WHERE id = ?

? bir placeholder; yerine yalnızca bir değer bağlanır.

DEĞİŞKEN

parameters = ("1 OR 1=1",)

type: tuple

Input SQL metnine eklenmez, parametre olarak bağlanır. İçindeki OR yeni bir koşul oluşturmaz.

LAB · PARAMETERIZED QUERY

Aynı input, parameterized query



id değeri '1 OR 1=1' olan kayıt yok: HTTP 200 ve boş liste.

LAB · PARAMETERIZED QUERY

Normal request hâlâ çalışıyor



Parameterized sürüme id=1: yine yalnız Alice.

Dört request, aynı veri

09 / 09 **Normal ve değiştirilmiş input'u iki sürüme gönder**

Sürüm	id	Result set	Response body
Concatenation	1	Alice	1 kayıt
Concatenation	1 OR 1=1	Alice + Bob	2 kayıt
Parameterized	1	Alice	1 kayıt
Parameterized	1 OR 1=1	Eşleşme yok	[]

Bu sorgudaki SQL injection kapandı. Uygulamadaki diğer sorgular ve authorization ayrıca kontrol edilir.

Sonuç her zaman response’ta görünmez

IN-BAND · UNION

Ek satırlar

Yanıtta beklenmeyen veri

BOOLEAN-BASED BLIND

Var / yok farkı

AND 1=1 ↔ AND 1=2

TIME-BASED BLIND

Süre farkı

Koşula bağlı gecikme

SECOND-ORDER

Daha sonra çalışır

Kaydet → başka sorguda kullan

SQL INJECTION · ORM VE PLACEHOLDER

Sürücü değişse de ayrım aynı

models.py · Django

```python
# Django: raw SQL still needs params
sql = "SELECT * FROM users "
sql += "WHERE name = %s"
User.objects.raw(sql, [name])

# Do not format with % or f-strings.
```

API / driver	Placeholder
Python sqlite3	?
Java JDBC	?
Node pg	$1
Django raw()	%s

ORTAK KURAL

Sorgu metni + ayrı parametre listesi

Placeholder'ı tırnak içine almayın; değeri manuel escape edip birleştirmeyin. ORM içinde raw SQL yazarken de driver'ın parametre API'sini kullanın.

Django: raw SQL ve parametreler · SQLite: value binding

SQL INJECTION · IDENTIFIER

ORDER BY kolonu parametre olamaz

Arayüz

Ada göre / Fiyata göre

→

HTTP parametresi

sort=price

→

Server mapping

"price" → sabit kolon adı

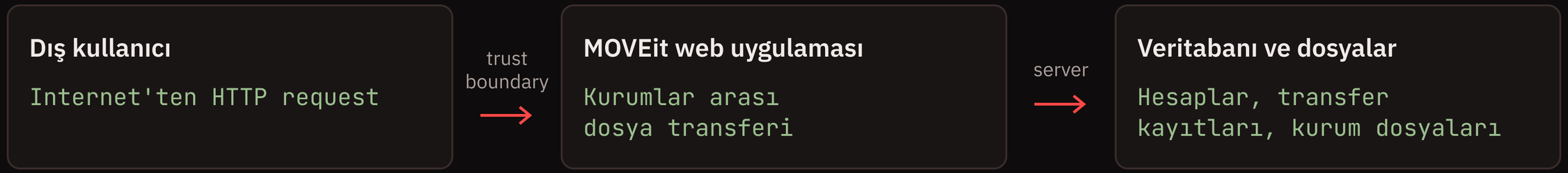
app.py · allowlist

```
1  columns = {"name": "name", "price": "price"}
2  key = request.args.get("sort", "name")
3  if key not in columns:
4      abort(400)
5  column = columns[key]
6  query = "SELECT name, price FROM products ORDER BY " + column
```

? yalnızca bir değerin yerine geçer; kolon veya tablo adı (identifier) için kullanılamaz. Bu durumda allowlist kullanılır.

VAKA · MOVEit TRANSFER · 2023

Internet’e açık bir SQL injection



CVE-2023-34362

SQL injection

Authentication gerektirmeyen HTTP request'lerle veritabanına yetkisiz erişim sağlanabiliyordu.

OLAYDA GÖZLENEN

LEMURLOOT web shell

CISA/FBI'ye göre CL0P grubu Mayıs 2023'te açığı kullanarak LEMURLOOT web shell'ini yerleştirdi ve veri sızdırdı.

Diyagram ürünün sadeleştirilmiş hâlidir. Kaynak: CISA/FBI AA23-158A

BÖLÜM 2

Command injection

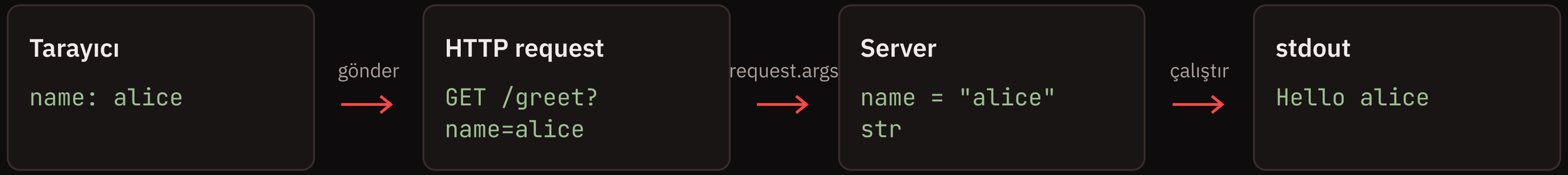
Kullanıcı input'u shell'e giden komut metnine eklenirse ne olur?



COMMAND INJECTION · POSIX

Komut sunucuda çalışıyor

01 / 05 Örnek: girilen adı selamlayan bir endpoint



NEREDE?

Sunucunun işletim sisteminde

shell=True ile komut metnini /bin/sh yorumlar. /usr/bin/echo aldığı argümanları stdout'a yazar.

NE DÖNER?

stdout response'a yazılır

Uygulama stdout'u yakalar ve text/plain olarak döndürür.

Selamlamak için bir OS komutuna gerek yok; mekanizmayı göstermek için en basit komut seçildi.

COMMAND INJECTION · NORMAL REQUEST

Input shell’e nasıl ulaşıyor?

02 / 05 name → command → shell → stdout → HTTP response

app.py · vulnerable

```
1  @app.get("/greet")
2  def greet():
3      name = request.args["name"]
4      command = "/usr/bin/echo Hello " + name
5      p = subprocess.run(command, shell=True,
6                         capture_output=True, text=True)
7      return Response(p.stdout, mimetype="text/plain")
```

name

`"alice"`

→

command

`"/usr/bin/echo Hello alice"`

→

p.stdout → HTTP 200

`"Hello alice\n"`

COMMAND INJECTION · INPUT'U DEĞİŞTİR

; ile ikinci bir komut

03 / 05 Kod aynı; yalnız name değişti

Request ve shell'e giden komut

```
GET /greet?name=alice%3B%20echo%20DEMO HTTP/1.1

# Decoded input: name = "alice; echo DEMO"
/usr/bin/echo Hello alice; echo DEMO
```

Shell ; karakterini ayırır

İki ayrı komut

→

1. komut

/usr/bin/echo
Hello alice
→ Hello alice

→

2. komut

echo DEMO
→ DEMO

→

HTTP 200 body

Hello alice
DEMO

Input artık yalnızca bir ad değil; shell'in kaç komut çalıştıracağını belirliyor. DEMO yerine herhangi bir komut yazılabilir.

COMMAND INJECTION · FIX

Program ve argümanlar ayrı

04 / 05 Komut metni yerine argüman listesi

app.py · fix

```
1  name = request.args["name"]
2  args = ["/usr/bin/echo", "Hello", name]
3  p = subprocess.run(args, shell=False,
4                     capture_output=True, text=True)
5  return Response(p.stdout, mimetype="text/plain")
```

Program

`/usr/bin/echo`

exec →

argv[1]

`Hello`

argüman →

argv[2] (tek parça)

`alice; echo DEMO`

Shell çalışmaz; ; karakteri ikinci bir komut başlatmaz. Python subprocess: security considerations

COMMAND INJECTION · KARŞILAŞTIRMA

İki sürüm, iki input

05 / 05 Normal davranış aynı; ikinci komut çalışmıyor

name	String + shell=True	Liste + shell=False
alice	Hello alice	Hello alice
alice; echo DEMO	Hello alice DEMO	Hello alice; echo DEMO

İkinci komut yok; ; artık adın bir parçası.

BÖLÜM 3

Path traversal

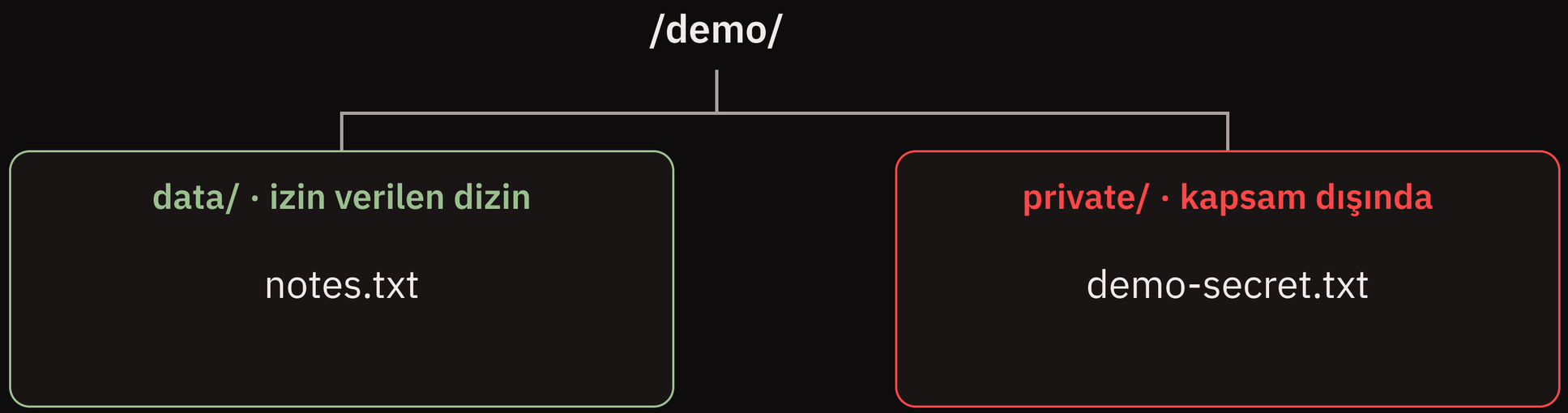
Uygulama bir dosya adı bekliyor. Input hangi dosyaya ulaşıyor?



PATH TRAVERSAL · DİZİN YAPISI

Yalnız data/ dizini sunulmalı

01 / 03 Normal kullanım ve base directory

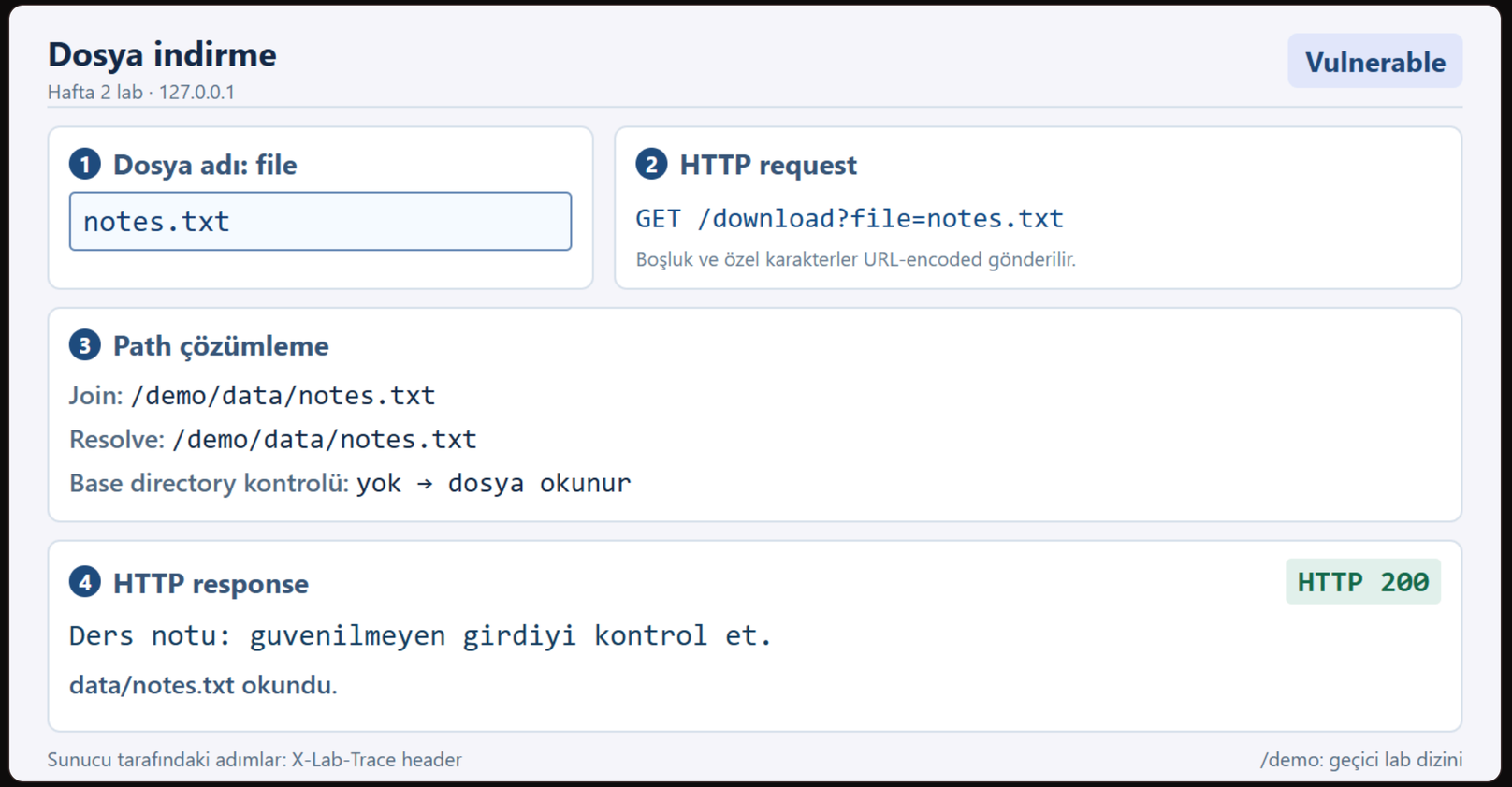


request.http

```
1 GET /download?file=notes.txt HTTP/1.1
```

LAB · NORMAL REQUEST

file=notes.txt



file → data/notes.txt → dosya içeriği → HTTP 200.

PATH TRAVERSAL · INPUT'U DEĞİŞTİR

../ bir üst dizine çıkar

02 / 03 Aynı parametre, farklı path

```
HTTP: GET /download?file=../private/demo-secret.txt
```



app.py

```python
filename = request.args["file"]
target = (download_root / filename).resolve()
content = target.read_text(encoding="utf-8")
return content, 200, {"Content-Type": "text/plain; charset=utf-8"}
```

resolve() path'i normalize eder; dosyanın base directory içinde olup olmadığını kontrol etmez.

LAB · PATH TRAVERSAL

data/ dışındaki dosya okundu



private/demo-secret.txt içeriği HTTP response body'sinde.

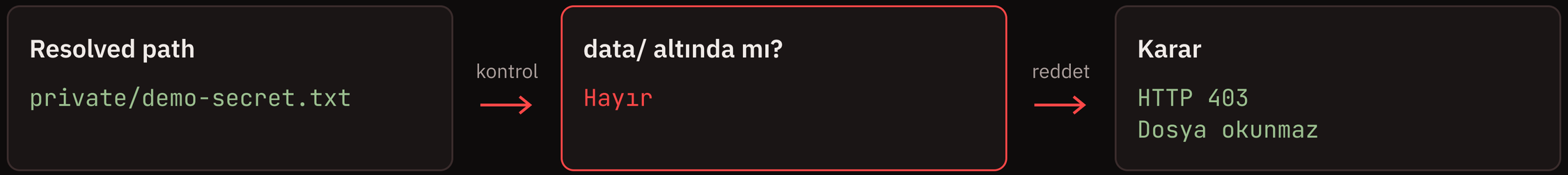
PATH TRAVERSAL · FIX

Resolve et, base directory’yi kontrol et, sonra oku

03 / 03 Dosya açılmadan önce karar ver

app.py · /safe/download

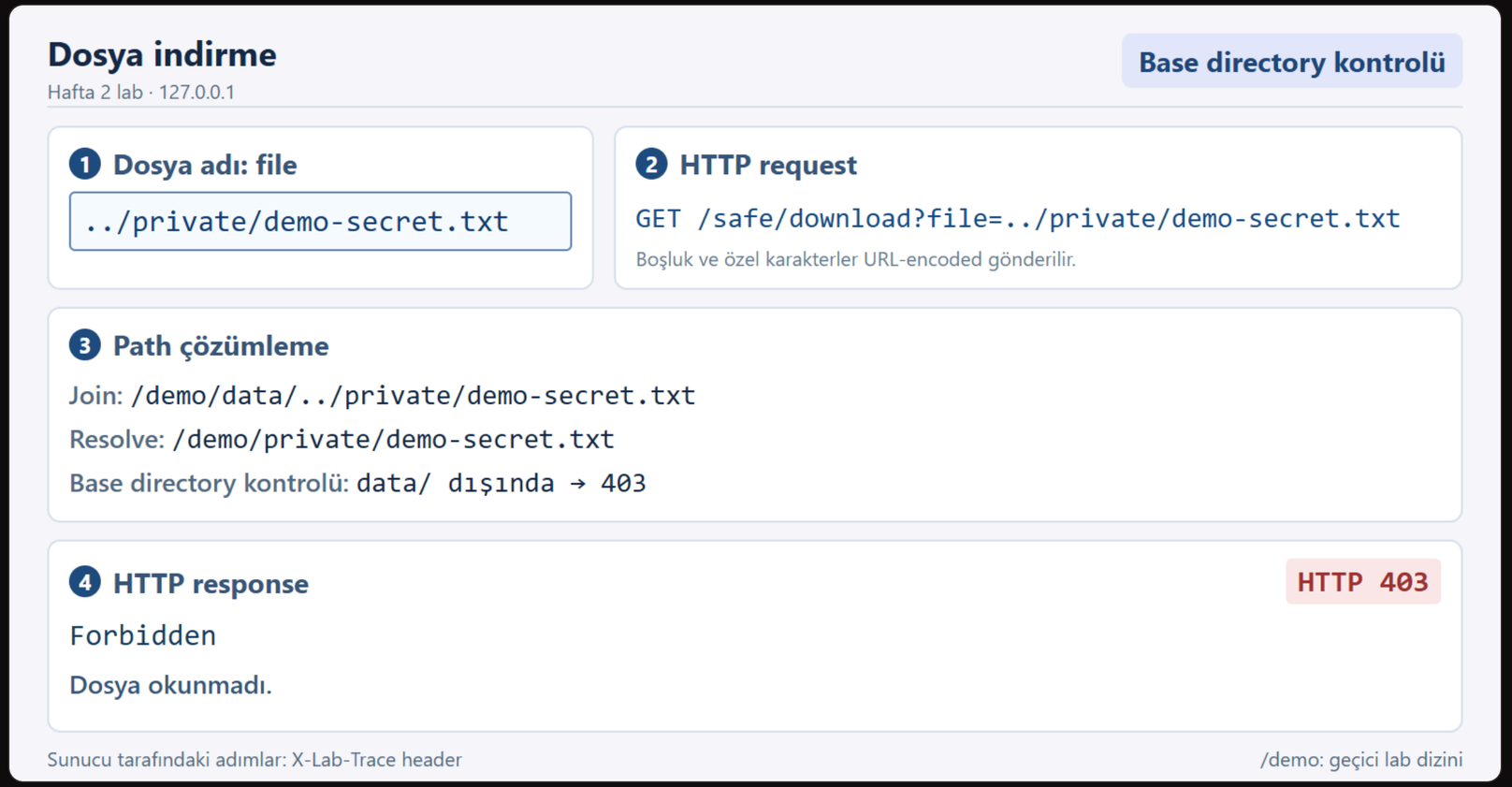
```
1  target = (download_root / filename).resolve()
2  if not target.is_relative_to(download_root):
3      return "Forbidden", 403
4  content = target.read_text(encoding="utf-8")
```



download_root da önceden resolve edilmiş. Kontrol string prefix'e değil path bileşenlerine göre yapılır.

LAB · FIX

Aynı request 403 alıyor



Aynı file değeri → base directory kontrolü → HTTP 403 Forbidden.

TEKRAR

Hangi satırı değiştirirdin?

SQL

```
1  db.execute("SELECT * FROM users WHERE id = " + uid)
```

Komut

```
1  subprocess.run("echo Hello " + name, shell=True)
```

Dosya okuma

```
1  target = (download_root / filename).resolve()
2  content = target.read_text(encoding="utf-8")
```

SORU · CEVAP

Sorular?



BÖLÜM EK

Ekstra konular

Aynı kalıbın diğer yorumlayıcılarda görünümü: argument injection, SSTI, NoSQL injection, XXE.



EK · ARGUMENT INJECTION

Input bir option olarak okunursa?

app.py · vulnerable

```
# Goal: show a file's Git history
path = request.args["file"]
args = ["git", "log", "-1",
        "--format=%s", path]
# shell=False, but path may be
# interpreted as a Git option.
```

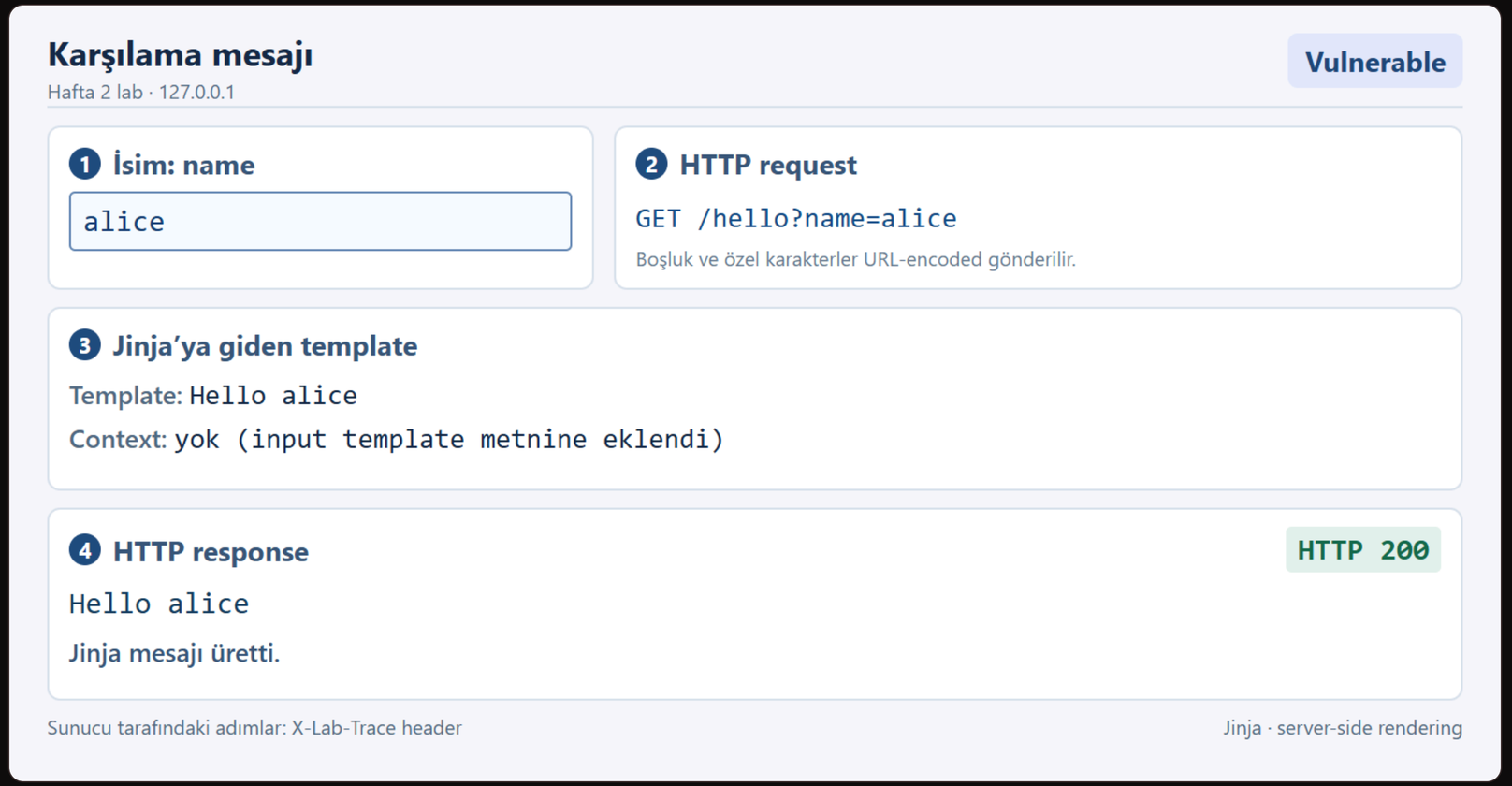
app.py · fix

```
# End options before the path
args = ["git", "log", "-1",
        "--format=%s", "--", path]
p = subprocess.run(args,
    shell=False, capture_output=True,
    text=True, cwd=REPO)
```

file	-- olmadan	-- ile
notes.txt	Dosyanın son commit mesajı	Dosyanın son commit mesajı
--output=demo.txt	Git çıktıyı demo.txt'ye yazar	Bu adda bir path aranır; bulunamaz

Shell yok; option'ı Git'in kendisi parse ediyor. -- desteği programa bağlıdır. git log

Normal request: name=alice



GET /hello?name=alice → Hello alice.

Input template’in parçası olursa

01 / 03 Oluşan template metni

app.py · /hello

```
1  name = request.args["name"]
2  return render_template_string("Hello " + name)
```

Input

`{{ 7*7 }}`

concat →

Template

`Hello {{ 7*7 }}`

render →

Jinja çıktısı

`Hello 49`

Jinja {{ ... }} içini expression olarak çalıştırır. Input template kaynağına eklenmiş durumda.

EK · SSTI · LAB

Gönderilen {{ 7*7 }}, çıkan 49



Template expression çalıştı.

EK · SSTI · FIX

Sabit template, ayrı context

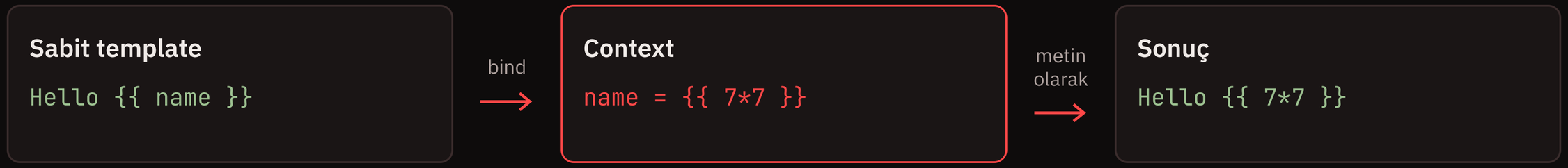
02 / 03 Template dosyası ve veri ayrı

hello.html

```
1 Hello {{ name }}
```

app.py · /safe/hello

```
1 name = request.args["name"]
2 return render_template(
3     "hello.html", name=name
4 )
```



Context değeri ikinci kez template olarak değerlendirilmez. HTML escaping ayrı bir konu (Hafta 3, XSS).

EK · SSTI · LAB

Aynı input metin olarak basılıyor



Fix'li sürüm: Hello {{ 7*7 }}.

EK · NoSQL INJECTION · NORMAL REQUEST

JSON alanı sorguya nasıl giriyor?

01 / 03 SKU koduna göre ürün arama

request.http

```
POST /products HTTP/1.1
Content-Type: application/json

{"sku":"P100"}
```

app.js · POST /products

```
// express.json() parses the body
const sku = req.body.sku;
const filter = {sku: sku};
const rows = await products.find(filter,
  {projection:{_id:0}}).toArray();
res.json(rows);
```

JavaScript değişkeni

```
sku = "P100"
string
```

→

MongoDB filter

```
{"sku":"P100"}
```

→

HTTP 200 · JSON

```
[{"sku":"P100",
"name":"Notebook"}]
```

products collection'ında iki document var: P100 / Notebook ve P200 / Pen.

EK · NoSQL INJECTION · TİP DEĞİŞTİ

String yerine object

02 / 03 Aynı sku alanı, artık bir JSON object

request.http

```
POST /products HTTP/1.1
Content-Type: application/json

{"sku":{"$ne":""}}
```

SUNUCUNUN KURDUĞU FILTER

{"sku":{"$ne":""}}

sku artık string değil, object. MongoDB $ne'yi "not equal" operatörü olarak okur: SKU'su boş string olmayan tüm ürünler.

Document	sku ≠ ""?	Sonuç
P100 / Notebook	true	Seçilir
P200 / Pen	true	Seçilir

Aynı res.json(rows) artık iki ürünü döndürür. MongoDB $ne

EK · NoSQL INJECTION · FIX

Önce type kontrolü, sonra sorgu

03 / 03 Beklenen tipi doğrula; filter'ı sunucu kursun

app.js · fix

```
1  const sku = req.body?.sku;
2  if (typeof sku !== "string")
3    return res.status(400).json({error: "sku must be a string"});
4  const rows = await products.find({sku: sku},
5    {projection: {_id: 0}}).toArray();
6  res.json(rows);
```

Normal body

`{"sku":"P100"}`
→ HTTP 200, tek ürün

→

Object içeren body

`{"sku":{"$ne":""}}`
→ HTTP 400

→

Kontrolün yeri

Tip yanlışsa MongoDB sorgusu çalışmaz

typeof kontrolü bu alan için yeterli. Birden fazla alan varsa schema validation (ör. JSON Schema, zod) kullanılır.

EK · XXE

External entity bir dosyaya bağlanırsa

request.http

```
POST /note HTTP/1.1
Content-Type: application/xml

<!DOCTYPE note [
 <!ENTITY ext SYSTEM
  "file:///lab/message.txt">
]>
<note>&ext;</note>
```

KAVRAMLAR

XML, DTD, entity

Normal <note>Hello</note> girdisinin metni Hello olur. DTD belgeye tanım ekler; &ext; bir entity referansıdır. Bu request'te ext, içeriği DEMO-DATA olan bir dosyaya bağlanıyor.

1 · request.data

`DTD + &ext; içeren XML`

→

2 · XMLParser

`resolve_entities=True`
`→ dosyayı okur`

→

3 · root.text

`"DEMO-DATA"`

XXE, kullanılan parser'a ve ayarlarına bağlıdır. no_network=True tek başına yerel dosya okumayı engellemez. lxml XMLParser

EK · XXE · FIX

Bu endpoint DTD kabul etmiyor

app.py · POST /note

```
xml = request.data
parser = etree.XMLParser(resolve_entities=False,
                         load_dtd=False, no_network=True)
root = etree.fromstring(xml, parser)
if root.getroottree().docinfo.doctype:
    abort(400)
return Response(root.text or "", mimetype="text/plain")
```

XML input	Fix sonrası
<note>Hello</note>	HTTP 200 · Hello
DOCTYPE + external entity	Entity çözülmez; DOCTYPE var → HTTP 400

KAYNAKLAR

Birincil kaynaklar

COMMAND / ARGUMENT INJECTION

Shell ve argümanlar

Python subprocess
git log ve -- ayracı

SQL INJECTION

Parameterized query

SQLite value binding
Django raw SQL

NoSQL / XXE

Operatör ve parser davranışı

MongoDB $ne
lxml XMLParser

MOVEit

Olay raporu ve teknik analiz

CISA/FBI AA23-158A
Horizon3 deep dive

Bağlantılar HTML sunumda tıklanabilir.